

Failure (a crashing tool, and a bad description) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — both halves, the direct way

```
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

# --- Half 1: a crashing tool, made safe ---

@tool
def flaky_divide(a: int, b: int) -> str:
    """Divide a by b. Returns an error message string if b is zero."""
    try:
        result = a / b
        return str(result)
    except ZeroDivisionError as e:
        return f"Error: {e}"

model_with_tools = ChatOpenAI().bind_tools([flaky_divide])
response = model_with_tools.invoke("Use the tool to divide 10 by 0")

for call in response.tool_calls:
    result = flaky_divide.invoke(call["args"])
    print("Tool result:", result) # "Error: division by zero", not a crash

# --- Half 2: a vague description, sharpened ---

@tool
def vague_weather(city: str) -> str:
    """gets data"""
    return f"Sunny in {city}"

@tool
def precise_weather(city: str) -> str:
```

```

"""Get the current weather (temperature and conditions) for a named city."""
return f"Sunny in {city}"

def count_calls(weather_tool, runs: int = 5) -> int:
    model_with_tools = ChatOpenAI().bind_tools([weather_tool])
    called = 0
    for _ in range(runs):
        response = model_with_tools.invoke("what's the weather like")
        if response.tool_calls:
            called += 1
    return called

vague_hits = count_calls(vague_weather)
precise_hits = count_calls(precise_weather)
print(f"Vague description: called {vague_hits}/5 times")
print(f"Precise description: called {precise_hits}/5 times")

```

Expected output (example):

```

Tool result: Error: division by zero
Vague description: called 1/5 times
Precise description: called 5/5 times

```

This shows both halves working: an error string instead of a crash, and a measurable difference between a vague and a precise description. It reruns the whole trial from scratch for each half rather than reusing the ambiguity exercise's trial function — fine here, but real code should share that logic instead of duplicating it.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — the model's reaction to the error, and a swappable-description factory

```
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

# --- Half 1: a crashing tool, made safe ---

@tool
def flaky_divide(a: int, b: int) -> str:
    """Divide a by b. Returns an error message string if b is zero."""
    try:
        result = a / b
        return str(result)
    except ZeroDivisionError as e:
        return f"Error: {e}"

def demonstrate_safe_failure() -> None:
    model_with_tools = ChatOpenAI(model="gpt-4o-mini", temperature=0).bind_tools(
        response = model_with_tools.invoke("Use the tool to divide 10 by 0")

    for call in response.tool_calls:
        result = flaky_divide.invoke(call["args"])
        print(f"Tool call {call['args']} -> {result}")

    follow_up = model_with_tools.invoke(
        "Use the tool to divide 10 by 0, then tell me in one sentence what happen
    )
    print(f"Model's reaction to the error: {follow_up.content}")

# --- Half 2: a vague description, sharpened ---

def make_weather_tool(description: str):
    def get_weather(city: str) -> str:
        return f"Sunny in {city}"
    get_weather.__doc__ = description
    return tool(get_weather)
```

```
def count_tool_calls(weather_tool, prompt: str, runs: int = 5) -> int:
    model_with_tools = ChatOpenAI(model="gpt-4o-mini", temperature=0).bind_tools(
        [weather_tool]
    )
    called = 0
    for _ in range(runs):
        response = model_with_tools.invoke(prompt)
        if response.tool_calls:
            called += 1
    return called

def demonstrate_description_fix() -> None:
    prompt = "what's the weather like"

    vague_tool = make_weather_tool("gets data")
    precise_tool = make_weather_tool(
        "Get the current weather (temperature and conditions) for a named city."
    )

    vague_hits = count_tool_calls(vague_tool, prompt)
    precise_hits = count_tool_calls(precise_tool, prompt)

    print(f"Vague description: called {vague_hits}/5 times")
    print(f"Precise description: called {precise_hits}/5 times")

if __name__ == "__main__":
    demonstrate_safe_failure()
    print()
    demonstrate_description_fix()
```

Expected output (example):

```
Tool call {'a': 10, 'b': 0} -> Error: division by zero
Model's reaction to the error: The division failed because you can't divide by zero

Vague description: called 1/5 times
Precise description: called 5/5 times
```

Difference from Basic: `make_weather_tool()` builds both the "before" and "after" tools from the exact same function body, so the description is the *only* thing that changes between the two trials — this is what makes the comparison actually mean something, instead of accidentally comparing two tools that differ in more than one way. `demonstrate_safe_failure()` also shows the model's *reaction* to the error, not just the raw error string, which is the part that actually matters in production.

Neither version distinguishes an expected failure from a genuine bug in the tool's own code, and neither checks whether 5 runs is enough to trust — that's what Advanced adds.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — separating expected failures from unexpected bugs

```
import logging

from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

logger = logging.getLogger(__name__)

@tool
def flaky_divide(a: int, b: int) -> str:
    """Divide a by b. Returns an error message string if b is zero."""
    try:
        result = a / b
        return str(result)
    except ZeroDivisionError as e:
        # expected failure -- the exact error is safe and useful to show
        return f"Error: {e}"
    except Exception:
        # unexpected -- a real bug in this tool's own code, not a known
        # failure mode. Log the real thing, but don't leak it to the model.
        logger.exception("Unexpected error in flaky_divide")
        return "Error: something went wrong running this tool. It's been logged."

model_with_tools = ChatOpenAI(model="gpt-4o-mini", temperature=0).bind_tools([flaky_divide])

# the expected case
response = model_with_tools.invoke("Use the tool to divide 10 by 0")
for call in response.tool_calls:
    print(flaky_divide.invoke(call["args"]))

# simulating an unexpected bug (e.g. b is somehow the wrong type at runtime)
print(flaky_divide.invoke({"a": 10, "b": "not a number"}))
```

Expected output:

Error: division by zero

Error: something went wrong running this tool. It's been logged.

The first line is the expected, specific failure — safe to show as-is. The second line is a genuine bug (dividing by a string raises `TypeError`, not `ZeroDivisionError`) — it gets logged in full server-side via `logger.exception(...)` (which includes the real traceback), but the model only ever sees a generic, safe message. Without the split, both cases would return `str(e)` directly, and a real internal error message would go straight into the conversation indistinguishable from an expected one.

Approach 2 — a larger sample for the description comparison, reported as a percentage

```
def count_tool_calls_large(weather_tool, prompt: str, runs: int = 20) -> tuple[int, bool]:
    model_with_tools = ChatOpenAI(model="gpt-4o-mini", temperature=0).bind_tools([weather_tool])
    called = 0
    for _ in range(runs):
        response = model_with_tools.invoke(prompt)
        if response.tool_calls:
            called += 1
    return called, runs

def demonstrate_description_fix_at_scale() -> None:
    prompt = "what's the weather like"

    vague_tool = make_weather_tool("gets data")
    precise_tool = make_weather_tool(
        "Get the current weather (temperature and conditions) for a named city."
    )

    vague_hits, vague_total = count_tool_calls_large(vague_tool, prompt)
    precise_hits, precise_total = count_tool_calls_large(precise_tool, prompt)

    print(f"Vague: {vague_hits}/{vague_total} ({vague_hits / vague_total:.0%})")
    print(f"Precise: {precise_hits}/{precise_total} ({precise_hits / precise_total:.0%})")
```

Expected output (example):

Vague: 3/20 (15%)

Precise: 19/20 (95%)

An 80-point swing at 20 runs is a far stronger claim than a 4/5-vs-1/5 swing at 5 runs — both "look dramatic," but only one of them is actually backed by enough data to write down as a finding.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate catches every exception the same way and trusts one 5-run comparison. Approach 1 fixes Half 1's gap — an unexpected bug in the tool's own code no longer looks identical to an expected, known failure mode, and the real error is still visible to whoever's watching the logs. Approach 2 fixes Half 2's gap — a bigger sample means the before/after comparison is actually something you could put in front of someone else and defend.

Which one should you actually write? Approach 1's expected-vs-unexpected split is worth doing on any tool you expect to keep and reuse (which, per this document's Goal, is every tool in the Build Task's library) — it costs one extra `except` block and pays for itself the first time a real bug hides behind a generic error string. Approach 2's larger sample is worth the extra API calls once you're about to act on a description-fix result for real — for a quick sanity check during development, Intermediate's 5-run version is fine.